DAA EXPERIMENT 7

Selection Sort on Random, Reverse and Sorted Arrays

Aim

To implement Selection Sort on random, reverse, and already sorted arrays and analyze its complexity.

Algorithm

1. Create a random-order array, a reverse-order array, and an already sorted array.
2. For each array, find the minimum element in the unsorted part.
3. Swap the minimum element with the first element of the unsorted part.
4. Repeat until the array is sorted.
5. Display the result for each array.
6. Stop.

Program

```
# Selection Sort on Random, Reverse and Sorted Arrays

def selection_sort(arr):
    n = len(arr)

    for i in range(n):
        min_index = i

        for j in range(i + 1, n):
            if arr[j] < arr[min_index]:
                min_index = j

        arr[i], arr[min_index] = arr[min_index], arr[i]

    return arr

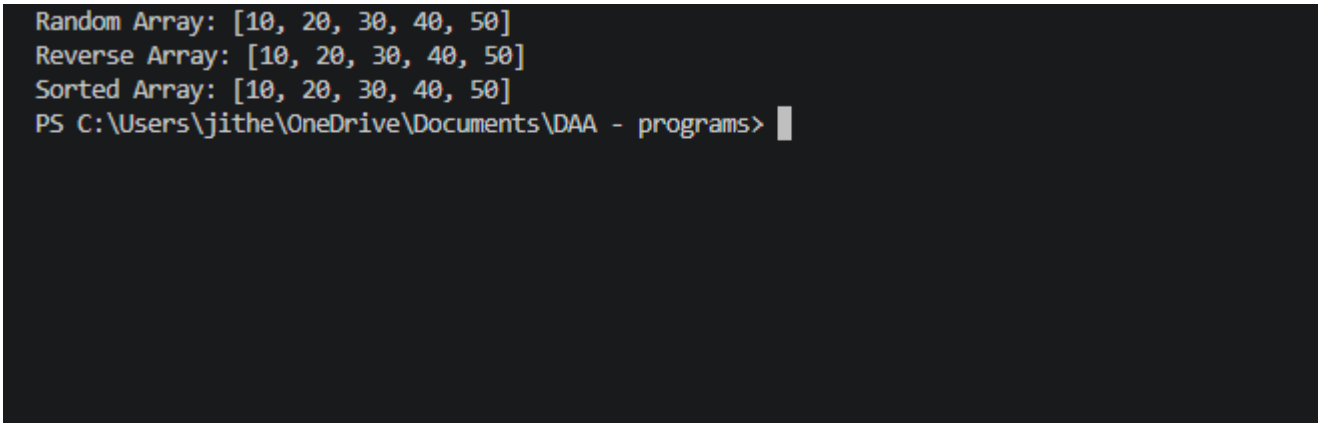

random_arr = [40, 10, 30, 20, 50]
reverse_arr = [50, 40, 30, 20, 10]
sorted_arr = [10, 20, 30, 40, 50]

print("Random Array:", selection_sort(random_arr))
print("Reverse Array:", selection_sort(reverse_arr))
print("Sorted Array:", selection_sort(sorted_arr))
```

Complexity Analysis

- Best Case: $O(n^2)$
- Average Case: $O(n^2)$
- Worst Case: $O(n^2)$
- Space Complexity: $O(1)$

Output



Result

Thus, Selection Sort was tested on random, reverse, and sorted arrays.